

Estimating the Live Memory Footprint of a Program over Time from Sparse Dynamic Instrumentation

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

MemPrint estimates the total memory footprint of a program from a sparse, randomly sampled Pin trace by learning a correction factor from fully traced small inputs. We extend the question from one number per run to the live footprint over the whole run, with deallocations removed, and we report what we learned while answering it. First, estimators that work from sampled memory references cannot separate new memory from memory that is touched again, and at a 1-in-25 sampling rate the best of them, a known-rate species-richness estimator with a learned correction, still has 24% mean error when extrapolating to a larger input. Second, sampling addresses instead of references removes that difficulty, because every address is selected with the same probability regardless of how often it is accessed. Address sampling needs no training, and it estimates the live footprint over time within 0.9% to 4.2% mean error on PolyBench at 1 in 25 to 1 in 100 addresses, and within 0.2% to 6.1% on miniVite with 1, 4 and 16 threads at 1 in 25 to 1 in 1000. Third, address sampling still pays for a check on every memory access, which costs $20\times$ to $42\times$ native time on large inputs. We therefore watch accesses only in short windows and track allocations and page residency for the whole run. Reused heap blocks are measured by the pages written since they were allocated, from the kernel’s soft-dirty bits, and a density measured per block on sampled 64-byte chunks corrects for overlapping accesses. Watching 5% of the run, this windowed estimator stays within 0.2% of the true footprint on PolyBench LARGE, within 3.1% to 3.8% mean error (1.9% to 3.0% at the peak) on miniVite with 65536 vertices, and within 24% on a heap-churn test on which page residency alone is 248% too high, at $2.8\times$ to $3.2\times$ native time on PolyBench and $13\times$ to $15\times$ on miniVite.

1 Introduction

A compiler or runtime that places, prefetches or compresses data needs to know how much memory a program touches and when. Heap profilers such as Valgrind Massif [15] record allocations, but an allocation says nothing about which of its bytes are used, and they miss stack and library memory. Dynamic binary instrumentation with Pin [8] sees every memory access, but instrumenting every access slows a program by one to two orders of magnitude. MemPrint [10] addresses the cost by sampling memory references sparsely and learning, from fully traced small inputs, a factor α that scales the sampled footprint to the true footprint of a large input.

MemPrint predicts one number per run, the cumulative footprint. Many uses need the footprint over time instead, with freed memory removed, because the peak of the live footprint determines how much memory a program needs and its timing determines when that memory is needed. In this work we extend MemPrint to estimate the live footprint over time, and we record the approaches we tried, including the ones that failed, because the failures explain the design that works.

In this work, we make the following contributions:

- We extend the MemPrint Pin tool with deallocation tracking, timed snapshots and partial traces, and we identify and fix two cross-thread ordering errors that made multithreaded peaks up to 19% too low (Section 3).
- We show that estimators working from sampled memory references, including the per-snapshot α model, Chao1, iChao1 and a known-rate negative binomial estimator with a learned correction, cannot reach low error at sparse rates, because a sparse reference sample cannot tell new memory from reused memory (Section 4).
- We propose address (spatial) sampling, which selects addresses by a salted hash and records every access to them, and show that it estimates the live footprint over time without training (Section 5).
- We propose windowed sampling, which watches accesses only in short windows while tracking allocations, page residency and, for reused heap blocks, the pages written since allocation for the whole run, and which corrects for overlapping accesses with a per-block density measured on sampled 64-byte chunks (Section 6).
- We evaluate all estimators against full traces on PolyBench [14] and miniVite [6], and list the limitations and next steps (Section 7 and Section 9).

2 Background

Footprint. Following MemPrint, the *footprint* of a set of memory references is the sum, over the distinct start addresses referenced, of the largest access size seen at each start address. The *live footprint* at time t is the footprint of the references made up to t , with every address inside a freed range removed at the moment it is freed. Time is measured in memory references executed, because that unit does not depend on how much the instrumentation slows the program.

MemPrint. The MemPrint Pin tool has two modes. The *splitter* records every memory reference and, for a list of sampling intervals k , splits a 1-in- k subsample of the references into 20 independent bins. The *sampler* records 1 in i references and assigns each sampled reference to a Poisson number of bins out of s , with mean $\lambda = s/r$. MemPrint fits a log-linear regression for $\alpha = F/\bar{F}_{\text{bin}}$, where F is the true footprint and $\bar{F}_{\text{bin}}$ the mean footprint of the bins, from seven features of the bins (their mean, their standard deviation and the sampling interval), on the splitter traces of small inputs, and applies it to a sampler trace of a large input. We reorganized the original scripts into one package and reproduced all 16 data figures of the MemPrint manuscript pixel for pixel, together with its tables. The only difference is the manuscript’s average row of the MAPE table, which was computed from an earlier version of the data.

Workloads and machine. We use four PolyBench/C 4.2.1 kernels [14] (2mm, gemm, jacobi-2d and atax), compiled without optimization as in MemPrint, at the sizes MINI to MEDIUM for the reference and address sampling studies and at LARGE for windowed sampling. We use miniVite [6], a distributed Louvain community detection proxy application, on generated random geometric graphs of 1024 to 65536 vertices, built against OpenMPI 5.0.5 from Spack and run as one MPI rank with 1, 4 or 16 OpenMP threads and `OMP_WAIT_POLICY=passive`. All runs use Pin 3.30 on a two-socket Intel Xeon E5-2620 v4 machine (16 cores, 32 threads, 62 GB) with Rocky Linux 8.10 (kernel 4.18) and GCC 8.5.

Error metrics. For an estimated curve $\hat{F}(t)$ and the true curve $F(t)$ from a full trace of the same input, we report three metrics, each as a percentage. The *mean error* is the mean absolute percentage error over the snapshots, with times compared as fractions of each run. The *error of the peak* compares $\max_t \hat{F}(t)$ with $\max_t F(t)$, wherever each occurs. The *error at the peak* compares $\hat{F}$ and F at the moment F peaks. A trained estimator is evaluated, as in MemPrint, on the largest input (extrapolation) and on the middle input (interpolation) after training on the other inputs.

3 Instrumenting deallocations and time

We added three options to the Pin tool, and with all three off its outputs remain byte-identical to the original tool, which we verified on PolyBench with fixed seeds and address space randomization disabled.

Deallocation tracking (-track_frees). The tool hooks the malloc family (malloc, calloc, realloc, memalign, aligned_alloc, valloc and posix_memalign) and records each block’s size at the allocator’s exit, and it removes a block’s range from every footprint when the block is freed. Three details were necessary for correct results. First, glibc’s **free** leaves through a tail jump that Pin does not report as a return, so the tool releases a block at the entry of **free**. Second, only the outermost allocator call is tracked, because allocator functions call each other, and the tool skips PLT stubs and functions that share an address (aliases), which otherwise produce several entry events per call. Third, **munmap** is caught at the system call, so that mappings released by libraries that bypass libc are removed too. In buffered modes, a release is queued with the position of the thread’s trace buffer at the time of the call, so that it is applied after the thread’s earlier accesses.

Snapshots and partial traces (-snapshot, -stop). Every N memory references the tool appends the current footprints to a timeline file, and **-stop** writes all outputs after N references and detaches Pin, so that the program finishes natively. Formatting the snapshot rows with the C runtime that Pin provides took about 7 ms per snapshot, so the tool formats numbers by hand.

Cross-thread ordering. Pin buffers each thread’s references separately, so references reach the tool in buffer order rather than program order. As a result, a free in one thread could be applied before another thread’s earlier references to the same block, and those references then added the freed block back. On a test with 8 threads and 12 MB live at a barrier, the full trace reported a peak of 9.8 to 10.8 MB. The tool now uses 16-page buffers when snapshots or frees are on, which gives 12.08 MB on every run. For address sampling (Section 5) we first held releases back with vector clocks, which was correct but took 7474 s on miniVite, because miniVite frees memory constantly while its MPI helper threads rarely fill a buffer. We replaced it by recording selected accesses immediately under a lock, which gives 12.03 to 12.27 MB on the test (truth 12.09 MB) in 28 to 40 s on miniVite. A test suite of small programs with known footprints (frees, realloc in place and moved, calloc and C++ new, partial munmap, threads, many small blocks) checks every mode.

4 Estimating from sampled references

The first approach applies the MemPrint idea at every snapshot: estimate the footprint at time t from the sampled references up to t . Table 1 summarizes the estimators, and Figure 1 shows them on 2mm MEDIUM.

The α model per snapshot. Fitting α on the splitter’s bins at every snapshot of the training inputs and applying it to the sampler gives 45% (extrapolation) and 31% (interpolation) mean error. A single bin holds 0.1% to 10% of the footprint and almost never samples an address twice, so the model cannot tell new memory from memory that is touched again, and on a plateau of the true footprint its estimate keeps rising.

Species-richness estimators. Counting how many addresses were sampled exactly once (f_1), twice (f_2), and so on, is the classic setting of estimating the number of unseen species, and the tool now reports f_1 to f_4 and the union of each interval’s bins. Chao1 [3] and iChao1 [4] fix the plateaus, and the jacobi-2d extrapolation error falls from 42% to 52% down to 4% on the splitter’s unions and 9% on the sampler, but they are lower bounds when reuse is uneven, and they underestimate kernels with many addresses touched once by 30% to 60% at moderate density and overshoot by 50% to 245% at the densest unions, where $f_2 \ll f_1$.

A known-rate estimator. Unlike the ecological setting, our sampling rate p is known exactly, and so is the number of references executed, T . We model the number of accesses of an address as $1 + X$ with X negative binomial (mean m , shape a), so the number of times an address is sampled has the probability generating function $(q + pz) B(z)$ with $q = 1 - p$ and $B(z) = ((1 + c) - cz)^{-a}$, $c = mp/a$. For a given shape a , the number of addresses U follows from $U(1 + m) = T$ and $U(1 - P(0)) = S$, where S is the number of addresses seen, and the shape is chosen by how well $UP(1), \dots, UP(4)$ match f_1 to f_4 . On dense samples (1 in 5) its bias is +1% to -12%, against +53% to +245% for iChao1, but at 1 in 50 and sparser the counts f_2 to f_4 are too small to identify the shape, and the error ranges from -90% to +215%. Fixing the shape from the training inputs does not work (40% to 150% error), because the fitted shapes are small (0.02 to 0.08), which indicates a mixture of memory touched once and heavily reused data rather than one smooth distribution. We also found that a sampler’s union of bins holds only the fraction $1 - e^{-\lambda}$ of the sampled references, so its sampling rate is $(1 - e^{-\lambda})/i$ rather than $1/i$, and our first known-rate results had overstated the rate by $1.6\times$.

A hybrid estimator. The known-rate estimator errs systematically, underestimating while addresses have not yet been reused. We therefore fit a ridge regression for $\log(F/\hat{F}_{\text{known}})$ on the training inputs, from the known-rate extrapolation factor, $\log((f_2 + 1)/(f_1 + 1))$, f_1/S , the sampling rate and the α model’s features. With a dense sampler (1 in 3, about 40% to 55% of the cost of a full trace) the hybrid reaches 6% and 5% mean error, and at 1 in 25 it reaches 24% and 9% (Table 1). We tested four refinements at 1 in 25: training the correction at exactly the sampler’s rate, making the estimate non-decreasing between frees, fitting one reuse shape per run, and adding the share of new addresses among recent samples. The extrapolation error stayed between 23% and 25%, and replacing byte-valued features with size-invariant ratios did not reduce it either. The remaining error is a size bias: in 2mm and gemm each element is reused about N times, so a larger input shows more repeats per address at the same rate than any training input did.

5 Address sampling

The reference sampling estimators fail for one reason: an address accessed r times appears in a reference sample with probability $1 - (1 - p)^r$, and r is unknown and grows with the input size. Address sampling removes that dependence.

Table 1: Estimators that work from sampled references, on four PolyBench kernels with the largest (extrapolation) and middle (interpolation) MINI to MEDIUM size held out. Values are the mean absolute error over the kernels, in percent.

Estimator	Sampler	Extrapolation			Interpolation		
		Mean	Of peak	At peak	Mean	Of peak	At peak
α model per snapshot	1/25, 1/50	45	30	58	31	59	49
Chao1	1/25	28	9	33	57	39	60
iChao1	1/25	27	11	33	55	36	58
Known rate	1/3	12	26	17	7	23	10
Hybrid	1/3	6	11	12	5	9	8
Hybrid, with refinements	1/25	24	21	38	9	12	24
Address sampling (Section 5)	1/25	0.9	0.8	0.8	3.4	3.4	3.4

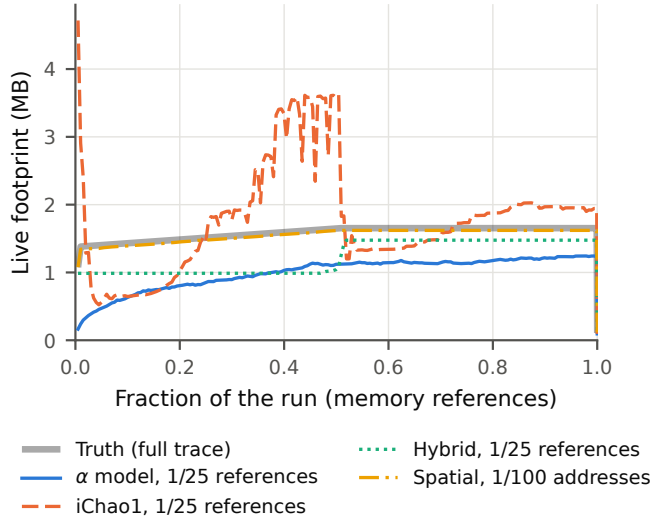


Figure 1: Live footprint of 2mm MEDIUM (held out) over the run, against estimates from 1-in-25 reference samples and from 1-in-100 address samples. The address sample tracks the truth closely, while the reference sampling estimators overshoot, undershoot or lag.

Method. The tool’s *spatial* mode selects an address a when $h(a \oplus \sigma) < 2^{64}/i$, where h is the splitmix64 finalizer and σ a per-run salt, and it records every access to a selected address. Each address is selected with probability $1/i$ regardless of its access pattern, so the selected footprint $F_S(t)$ gives the unbiased estimate $\hat{F}(t) = i F_S(t)$ with no model and no training, and its relative error is about $1/\sqrt{U/i}$ for U live addresses. The tool also splits the selected addresses into 20 hash buckets, each an independent $1/(20i)$ sample, which gives every snapshot a standard error. The salt makes the selection independent of strides and alignment. Selected accesses are rare, so the tool records them and applies releases immediately under one lock, which keeps cross-thread order exact.

Accuracy. On PolyBench, with the largest and middle MINI to MEDIUM sizes held out as before, address sampling gives 0.9% and 3.4% mean error at 1 in 25 addresses, 2.3% and 4.2% at 1 in 100, 2.3% and 9.9% at 1 in 250, and 6.7% and 17.5% at 1 in 1000 (Figure 2). The error follows

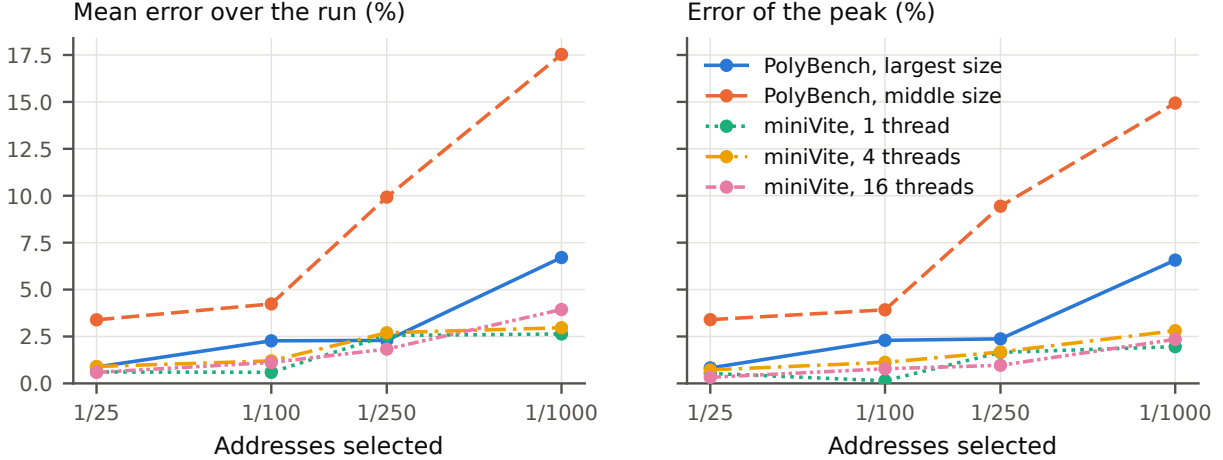


Figure 2: Address sampling error against the fraction of addresses selected. PolyBench values are the mean over 2mm, gemm, jacobi-2d and atax for the held-out largest and middle MINI to MEDIUM sizes. miniVite values are the mean over 1024, 4096 and 8192 vertices.

the binomial prediction: the largest sizes (130,000 to 200,000 addresses) stay within 1% to 7% down to 1 in 1000, whereas the middle sizes (about 28,000 addresses) need 1 in 100 or denser. The true footprint lies within two standard errors in 100% of snapshots at 1 in 25 and 1 in 100, and in 88% and 77% at 1 in 250 and 1 in 1000.

miniVite and threads. Built against the system OpenMPI, miniVite touched about 175 MB at every size, mostly MPI start-up buffers, and spinning OpenMP threads made 97% of its 1.76 billion references at 1024 vertices. Built against OpenMPI 5.0.5 with passive waiting, a full trace at 1024 vertices gives a cumulative footprint of 6.8 to 7.0 MB, which matches the 6.84 MB in the MemPrint manuscript, from 44 million references. With frees tracked, the live peak is 4.9, 6.2 and 7.9 MB at 1024, 4096 and 8192 vertices at every thread count. Address sampling gives 0.2% to 6.1% mean error at every rate from 1 in 25 to 1 in 1000 with 1, 4 and 16 threads, the error of the peak is at most 2.8%, and the truth lies within two standard errors in 94% to 100% of snapshots. At 8192 vertices it runs in 26 to 41 s, against 168 to 198 s for a full trace and about 0.25 s natively.

6 Windowed sampling

Address sampling still evaluates the selection test on every memory access. On the LARGE PolyBench inputs that test alone costs $20\times$ to $42\times$ native time (203 s for 2mm against 7.5 s), while Pin without a tool costs almost nothing (7.7 s). For long real workloads, therefore, the per-access test is the obstacle, and the sampling rate is not. Windowed sampling watches accesses only for W of every P references and keeps everything else that it needs running for the whole run at low cost (Figure 3).

Time between windows. Between windows the tool counts memory references with one inlined add per basic block, using the number of memory operands in the block. On miniVite with one thread this count matches the splitter’s per-access count to within 0.0001% over 1.78 billion references. A REP string instruction counts once rather than once per iteration.

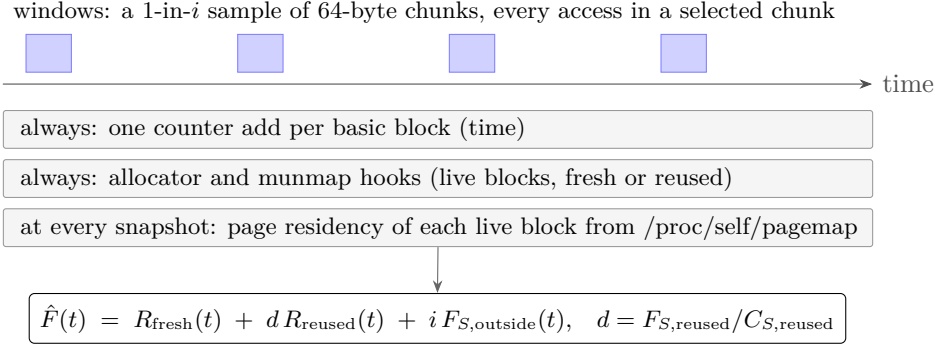


Figure 3: Windowed sampling. Accesses are watched only inside windows (shaded). Time, live allocations and page residency are tracked for the whole run. R is the resident (touched) bytes of fresh or reused blocks, F_S the selected footprint, C_S the bytes that selected accesses cover, and d the density.

Switching instrumentation. The tool switches between the two kinds of instrumentation with `PIN_RemoveInstrumentation`, which discards Pin’s translated code. However, translated code that is already running does not change, because a loop keeps jumping back into its own translation. In 2mm, a window that had closed kept sampling for the rest of the loop nest, so a run that should have watched nothing took 70s instead of 19s. Each thread therefore checks the window state every few thousand references, and when the state has changed it restarts at the current instruction with `PIN_ExecuteAt`, so that Pin translates the code again with the new instrumentation. The tool discards selected accesses that stale code records after a window has closed.

Live blocks and residency. The allocator hooks of Section 3 run for the whole run and maintain the set of live blocks, which also includes anonymous mmap’s made outside the allocator. A block is *fresh* when its pages were mapped for it, that is, an anonymous mmap or an allocation during which the allocator called `mmap` or `mremap`, and *reused* otherwise, for example a small block from the heap. A page of anonymous memory becomes resident at its first read or write, so the resident bytes of a fresh block, read from `/proc/self/pagemap` [16] at every snapshot, are its touched bytes to within one 4 KB page, including between windows. The machine enables transparent huge pages, which make a whole 2 MB region resident at one touch, so the tool disables them for the traced process with `prctl(PR_SET_THP_DISABLE)`. Without that setting, a test that touched 16 MB of a 64 MB block reported 18.4 MB resident, and with it the tool reports exactly 16 MB. The tool reads the pagemap entries in chunks of 512 pages and visits blocks in address order, so neighboring blocks share reads.

Written pages of reused blocks. The residency of a reused block can include pages that earlier blocks at the same addresses touched, because freed heap memory stays resident. On a heap-churn test in which every block touches only the first quarter of its memory (Section 6.1), residency over-estimates the footprint by 248%. The tool therefore measures a reused block by the pages written since it was allocated, from the kernel’s soft-dirty bits [16]: writing 4 to `/proc/self/clear_refs` clears them, and pagemap bit 55 then reports which pages have been written since. Heap memory is written before it is read, so a reused block’s written pages are its touched pages. Clearing the bits write-protects every page, so the next write to each page costs one fault, and the tool clears them only at three kinds of events, after recording the written pages of every live reused block. First, it clears them at the allocator’s entry for allocations of 16 KB or more, so that earlier writes to the new block’s pages do not count while `calloc`’s zeroing and `realloc`’s copy, which happen inside the

call, do. Second, it clears them around the `brk`, `mmap` and `mremap` system calls, because pagemap reports every page of a region that the kernel has just created or grown as soft-dirty, whether or not it was written, and without this step the written bytes of the churn test alternated between 1.3 MB and 3.7 MB. Third, it clears them every ten snapshots. Blocks whose pages are all marked written are skipped when recording, which reduced the runtime of miniVite with one thread from 138s to 86s.

Density. On miniVite, residency alone underestimates the peak by about 20% at every watched fraction (Figure 6), because the footprint counts the largest access at every start address, so overlapping accesses, for example unaligned copies, count more than the bytes they touch. In miniVite’s reused heap blocks, the footprint per touched byte, which we call the *density*, is about 1.45 for most of the run and about 1.1 at its end. A density measured on selected single addresses depends on how much of each block the windows saw, and a density measured on sampled pages was noisy (one run ended at 2.7), because a page with few touched addresses enters the sample only when one of them is selected. The windows therefore select 64-byte chunks instead of addresses. Inside a selected chunk the tool sees every access, so it knows both the footprint and the bytes covered on the same memory, and their ratio does not depend on the windows’ coverage of the run. Chunks are still selected with probability $1/i$, so $i F_S$ remains unbiased. The density is measured per block, once a block’s selected accesses cover $64\text{KB}/i$. Before that, a reused block uses the density of all reused blocks, and a fresh block uses 1, because the large arrays that fresh blocks hold are mostly written in whole words. A per-block density matters for fresh blocks too: in miniVite with 32768 vertices, the true peak occurs while a fresh block is written with overlapping accesses at a density of about 1.3, which the first version of the estimate counted at 1.

Estimate. At every snapshot the estimate is

$$\hat{F}(t) = \sum_{b \text{ live}} d_b(t) T_b(t) + i F_{S,\text{outside}}(t), \quad (1)$$

where T_b is the touched bytes of block b (resident bytes for a fresh block, written pages for a reused one), d_b its density, and $F_{S,\text{outside}}$ the selected footprint outside every block (stack, globals and shared memory), which only windows see. We first estimated reused blocks from the windows’ samples, with an allocation-site model for blocks that no window had yet examined, but that estimate missed every touch made between windows and reached 29% to 47% mean error on miniVite at 1% watched and 8% to 25% at 5%.

6.1 Evaluation

We compare windowed runs with a full trace (splitter with frees tracked) of the same input and thread count, at 1 in 100 chunks and about 20 windows per run, without tuning any setting per workload. The inputs are 2mm, gemm and jacobi-2d LARGE (live peaks of 27 to 37 MB, 28 to 49 billion references) and miniVite with 32768 and 65536 vertices (live peaks of 20 and 36 MB, 6.6 and 25 billion references). We also use a heap-churn test that keeps 64 live heap blocks of 16 to 112 KB, replaces the oldest at every step, and writes only the first quarter (or half) of each new block, so that new blocks land on pages that freed blocks touched. We compare (1) with three baselines: the resident bytes of all blocks without the density, the windows’ selected footprint alone ($i F_S$), and the live allocated bytes.

Table 2: Windowed estimate on miniVite, mean error / error of the peak in percent, 1 in 100 chunks.

Input, threads	Nothing watched	1%	5%	20%
65536, 1 thread	25.9 / -24.6	9.1 / -14.5	3.8 / -1.9	4.5 / -0.4
65536, 4 threads	26.4 / -24.8	8.4 / -14.4	3.1 / -3.0	1.7 / -2.8
32768, 1 thread	32.1 / -27.9	14.4 / -14.1	8.3 / -9.8	4.3 / -4.3
32768, 4 threads	32.3 / -28.0	17.1 / -15.2	5.7 / -9.0	2.1 / -4.8

PolyBench. Every PolyBench array is allocated by a fresh mmap, so residency alone gives the exact curve, and the windowed estimate is within 0.0% to 0.2% mean error and within 0.2% at the peak at every watched fraction, including none (Figure 5, left). The windows’ sample alone gives 0.6% to 8.3% mean error at 5% watched and 11% to 49% at 1%, because it misses memory that is written once and then not touched while a window is open. The live allocated bytes give 0.4% to 4.5%.

miniVite. The footprint of miniVite grows in reused heap blocks for the whole run, and it allocates far more than it touches, so the live allocated bytes overestimate it, with 67% to 107% mean error (Figure 5, right). Table 2 and Figure 6 show the windowed estimate. With 65536 vertices and 5% watched, it reaches 3.8% and 3.1% mean error with 1 and 4 threads, and -1.9% and -3.0% error of the peak, against 12.6% and 11.9% mean error and about -19% at the peak for residency without the density. At 1% watched the measured density is only 1.05 to 1.12, so the peak is about 15% low, and with nothing watched the density stays 1 and memory outside blocks is not seen at all. Selecting 1 in 25 chunks instead of 1 in 100 changes the results by less than 3 points.

The peak of miniVite with 32768 vertices. With 32768 vertices the true peak is a spike that lasts about 0.3% of the run, at 90.5%, when 4.3 MB of new heap blocks appear and the true footprint rises from 13.3 to 19.9 MB. With a single density for reused blocks and none for fresh blocks, the estimate at the spike was 17.6 MB with one thread and 17.5 MB with four, so the 12% to 17% peak error that we first observed with four threads did not come from threads. It did not come from too few windows either: three repeated runs agreed within 1.4 points, 40 or 80 shorter windows were slightly worse than 20, and watching the whole run at 1 in 16 chunks still missed the peak by 12%. The cause was the fresh block written at the spike with a density of about 1.3. With a density per block, the peak error is -9.0% and -4.8% at 5% and 20% watched with four threads (from -17.2% and -11.9%), and -9.8% and -4.3% with one thread. At 5% watched no window always overlaps the spike, so the density of the block written there is not always measured.

Heap churn. On the churn test, the resident bytes overestimate the footprint by 248% (mean error) when a quarter of each block is touched and by 91% when half is touched, because freed blocks’ pages stay resident (Figure 4). Measuring reused blocks by their written pages reduces the error to 24% and 13%, and the error is the same at every watched fraction (22% to 26% for a quarter touched), because written pages rather than windows carry the estimate. The remaining error is page granularity: each block’s touched prefix ends inside a 4 KB page, which adds up to 4 KB per block, or about 0.25 MB for 64 blocks.

Cost. Table 3 and Figure 7 report runtimes measured one at a time on an otherwise idle machine, one run per configuration. On PolyBench LARGE, windowed sampling at 5% watched costs $2.8 \times$

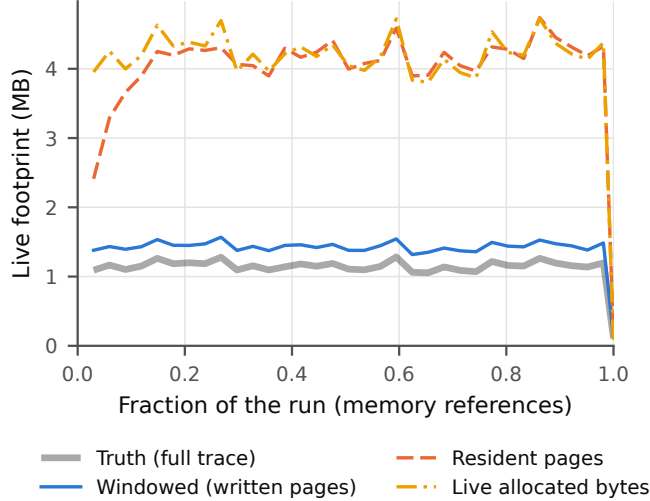


Figure 4: Heap churn with a quarter of each block touched, 5% of the run watched: resident pages count memory that freed blocks touched, whereas the pages written since each block’s allocation follow the truth to within page granularity.

Table 3: Runtime in seconds, one run each, 1 in 100 addresses or chunks, about 20 windows.

Workload	Native	Pin, no tool	Full spatial	Windowed, fraction watched			
				none	1%	5%	20%
2mm LARGE	7.5	8.1	211	15.8	16.9	23.1	43.6
gemm LARGE	6.0	6.1	215	11.7	13.1	19.0	37.0
jacobi-2d LARGE	11.3	11.6	236	21.5	23.3	31.2	59.4
miniVite 65536, 1 thread	6.5	11.5	163	59.6	79.4	84.3	104.1
miniVite 65536, 4 threads	6.1	11.0	161	72.3	85.8	93.0	111.2

to $3.2\times$ native time, against $21\times$ to $36\times$ for full address sampling, and with nothing watched it costs $1.9\times$ to $2.1\times$, which is mostly the cost of the per-block counter. On miniVite with 65536 vertices it costs $13.0\times$ (1 thread) and $15.3\times$ (4 threads) at 5% watched, against $25\times$ and $27\times$. The floor on miniVite is higher because Pin itself costs $1.8\times$ (MPI start-up code is translated once) and because miniVite makes 8 million malloc and free calls at 65536 vertices, each hooked and recorded under a lock, and on miniVite with 16384 vertices disabling the allocator hooks saved 7s of 22s. Soft-dirty tracking accounts for about 14% of the miniVite runtime at 5% watched and 4% to 14% on PolyBench. One timing of jacobi-2d at 5% watched took 173s and a repeat took 31.2s, and we report the repeat. The full traces that provided the truth took 1237 to 1717s, but they ran concurrently, so they are not directly comparable.

7 Discussion and limitations

What makes the estimate work. Each component of (1) addresses one failure we observed. Residency records first touches whether or not a window is open, which the windows’ sample alone cannot do, written pages remove the pages that a reused block inherits from freed blocks, which residency alone cannot do, and the density corrects the difference between the footprint metric

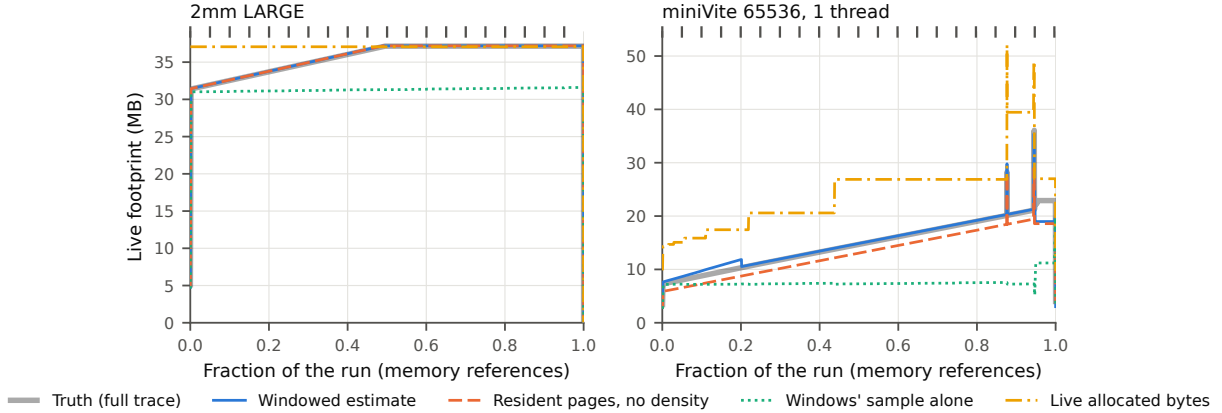


Figure 5: Live footprint over the run with 5% of the run watched (20 windows, marked at the top) and 1 in 100 chunks selected. On 2mm LARGE the windowed estimate and the resident pages coincide with the truth.

and touched bytes. The chunk sample makes the density independent of how much of the run the windows covered.

Limitations. A limitation of written pages is that they miss memory a reused block only reads, which is rare for heap memory, and that they are exact only to the page, which adds up to 4 KB per block and dominates for programs with many small, partly touched blocks (22% on the churn test). Blocks smaller than 16 KB do not clear the soft-dirty bits when they are allocated, so they can inherit marks from writes made since the last clear, and with several threads, writes made by other threads between the entry and the exit of a `brk` or `mmap` call are lost. A limitation of the density is that a block, or the set of reused blocks, needs a few windows’ worth of chunks before its density leaves the default, which explains the error at 1% watched and the -9% peak error on miniVite with 32768 vertices at 5% watched, whose peak lasts only 0.3% of the run. Memory outside blocks is seen only in windows, and the tool does not track `mremap` outside `realloc` or stack frames. Address and chunk selection depend on absolute addresses, which move when the tool binary changes size, so spatial outputs are not byte-identical across tool builds, whereas splitter and sampler outputs are.

Generality of the evaluation. We evaluated two applications and one synthetic heap-churn test, and PolyBench uses only fresh memory. It remains unclear how the windowed estimate behaves on real programs with heavy heap churn, on long-running programs whose footprint changes in phases shorter than a window period, and with many MPI ranks.

8 Related work

Valgrind [11] and its heap profiler Massif [15] record allocations at high cost and do not measure which allocated bytes are used. MemGaze [7] analyzes sampled load-level traces collected with hardware support, and PEBS [12] and ARM SPE [9] sample memory accesses at low cost, but like the sampler of MemPrint, they sample references, so they share the reuse bias of Section 4. StatCache-style locality profiling [1] and reuse distance analysis [2] estimate cache behavior rather than footprint. PinPoints [13] instruments representative regions of a run, and instrumentation

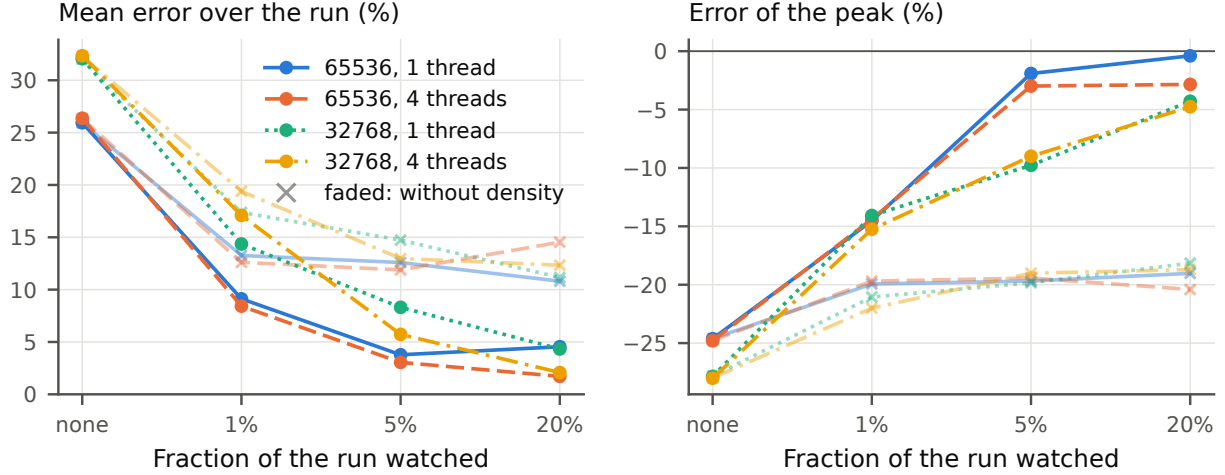


Figure 6: miniVite: windowed estimate error against the fraction of the run watched, with 1 in 100 chunks. Faded lines are the same estimate without the density correction.

sampling for datacenter profiling [5] switches instrumentation on and off over time, as our windows do, but neither reconstructs a footprint that depends on first touches made outside the instrumented regions. Chao1 [3] and iChao1 [4] estimate the number of unseen species from frequency counts, and our known-rate estimator adapts that idea to a sample whose rate is known exactly.

9 Next steps

We propose the following next steps, in order of priority.

1. **Place windows adaptively.** Open a window when a large block is allocated or when the rate of newly resident or written pages changes, instead of at a fixed period. The peak of miniVite with 32768 vertices lasts 0.3% of the run, and a window opened at the allocation that starts it would measure its density.
2. **Evaluate a real program with heavy heap reuse.** The churn test is synthetic. A graph or sparse-matrix code with per-iteration buffers, or an interpreter, would show whether written pages are enough in practice and how much the cost of clearing soft-dirty bits grows with the allocation rate.
3. **Reduce page granularity for small blocks.** For blocks smaller than a few pages, combine the written pages with the chunk sample, for example by scaling a block’s written bytes by the fraction of its selected chunks that were touched.
4. **Reduce the floor cost of the allocator hooks and the soft-dirty clears.** miniVite spends about a third of its windowed runtime in the hooks, and soft-dirty tracking adds about a fifth with one thread (8509 clears). Probe mode instrumentation of the allocator, or a lock-free per-thread log of allocation events merged at snapshots, would lower it.
5. **Evaluate at scale.** Run windowed sampling on a long application with several MPI ranks, where a full trace is infeasible, and validate the peak against the maximum resident set size reported by the operating system, which is an upper bound at page granularity.

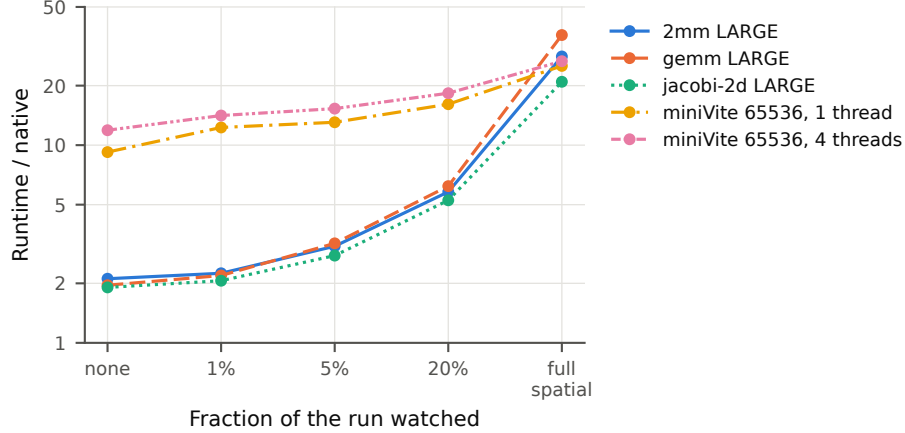


Figure 7: Runtime relative to native against the fraction of the run watched, with full address sampling for comparison (logarithmic scale).

6. **Combine with hardware sampling.** PEBS or ARM SPE could replace the per-block counter and the chunk sample at even lower cost, with the known-rate correction of Section 4 for their reference bias.
7. **Revisit forecasting.** Forecasting the rest of the curve from a prefix by scaling the training curves reached about 10% to 20% error of the peak for interpolation and was unreliable for extrapolation. Windowed sampling makes long prefixes cheap, which may make a phase-based forecast practical.

10 Conclusion

Estimating the live footprint over time from sampled memory references fails at sparse rates, because a reference sample cannot separate new memory from reused memory. Sampling addresses instead removes that difficulty and needs no training, and watching addresses only in short windows, while tracking allocations and page residency for the whole run, keeps the estimate within 0.2% on PolyBench LARGE and within 3.1% to 3.8% on miniVite with 65536 vertices at 5% of the run watched, at $2.8\times$ to $3.2\times$ native time on PolyBench and $13\times$ to $15\times$ on miniVite. Measuring reused heap blocks by the pages written since their allocation keeps the estimate within page granularity when freed memory is reused.

References

- [1] Erik Berg and Erik Hagersten. Fast data-locality profiling of native execution. In *Proceedings of the 2005 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, SIGMETRICS '05, page 169–180, New York, NY, USA, 2005. Association for Computing Machinery.
- [2] Kristof Beyls and Erik H. D’Hollander. Reuse distance-based cache hint selection. In *Proceedings of the 8th International Euro-Par Conference on Parallel Processing*, Euro-Par '02, page 265–274, Berlin, Heidelberg, 2002. Springer-Verlag.

- [3] Anne Chao. Nonparametric estimation of the number of classes in a population. *Scandinavian Journal of Statistics*, 11(4):265–270, 1984.
- [4] Chun-Huo Chiu, Yi-Ting Wang, Bruno A. Walther, and Anne Chao. An improved nonparametric lower bound of species richness via a modified Good–Turing frequency formula. *Biometrics*, 70(3):671–682, 2014.
- [5] Hyoun Kyu Cho, Tipp Moseley, Richard Hank, Derek Bruening, and Scott Mahlke. Instant profiling: Instrumentation sampling for profiling datacenter applications. In *Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 1–10, 2013.
- [6] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, and Assefaw H. Gebremedhin. miniVite: A graph analytics benchmarking tool for massively parallel systems. In *Proceedings of the IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems Workshop (PMBS)*, 2018.
- [7] Ozgur O. Kilic, Nathan R. Tallent, Yasodha Suriyakumar, Chenhao Xie, Andrés Marquez, and Stephane Eranian. Memgaze: Rapid and effective load-level memory trace analysis. In *2022 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 484–495, 2022.
- [8] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI ’05*, page 190–200, New York, NY, USA, 2005. Association for Computing Machinery.
- [9] Samuel Miksits, Ruimin Shi, Maya Gokhale, Jacob Wahlgren, Gabin Schieffer, and Ivy Peng. Multi-level memory-centric profiling on arm processors with arm spe. In *Proceedings of the SC ’24 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis, SC-W ’24*, page 996–1005. IEEE Press, 2025.
- [10] Nafis Mustakin, Joseph Manzano, and Daniel Wong. MemPrint: Constructing program memory footprint estimations using statistical methods from sparsely sampled Pin-based memory traces. Manuscript, 2025.
- [11] Nicholas Nethercote and Julian Seward. Valgrind: a framework for heavyweight dynamic binary instrumentation. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI ’07*, page 89–100, New York, NY, USA, 2007. Association for Computing Machinery.
- [12] Aleix Roca Nonell, Balazs Gerofi, Leonardo Bautista-Gomez, Dominique Martinet, Vicenç Beltran Querol, and Yutaka Ishikawa. On the applicability of pebs based online memory access tracking for heterogeneous memory management at scale. In *Proceedings of the Workshop on Memory Centric High Performance Computing, MCHPC’18*, page 50–57, New York, NY, USA, 2018. Association for Computing Machinery.
- [13] H. Patil, R. Cohn, M. Charney, R. Kapoor, A. Sun, and A. Karunanidhi. Pinpointing representative portions of large intel ® itanium ® programs with dynamic instrumentation. In *37th International Symposium on Microarchitecture (MICRO-37’04)*, pages 81–92, 2004.

- [14] Louis-Noël Pouchet. PolyBench/C 4.2.1: The polyhedral benchmark suite, 2016. <https://sourceforge.net/projects/polybench/>.
- [15] Julian Seward and Nicholas Nethercote. *Valgrind Massif: Heap Profiler*, 2015. Available at <http://valgrind.org/docs/manual/ms-manual.html>.
- [16] The Linux Kernel Documentation. Examining process page tables (/proc/pid/pagemap), 2024. <https://www.kernel.org/doc/html/latest/admin-guide/mm/pagemap.html>.